CS 202: Project 2 Report

Online Food Ordering System

Group 5

Goktug Gokbulut & Alperen Cimen

Spring 2026

1 Introduction

The Online Food Ordering System is a full-stack application that allows customers to browse restaurants, apply discount coupons, place orders, track order status, and rate restaurants after order acceptance. Restaurant managers, the second user type supported by the system, can register and manage their restaurants, define menu categories and menu items, create promotional coupons, accept incoming orders, and review monthly sales statistics. The system is implemented as a Java Spring Boot backend with a Java user interface, communicating with a MySQL relational database through manually-written SQL queries over JDBC. ORM frameworks and automatic query generators were not used; all data definition, manipulation, and retrieval is expressed in explicit SQL by hand.

This report documents the design and implementation of the system. Section 2 presents the entity-relationship diagram. Section 3 enumerates the functional dependencies derived from the schema. Section 4 explains the normalization of the resulting tables up to third normal form. Section 5 records the principal design decisions and constraint enforcement strategies. Section 6 describes the main SQL queries used by the application. Section 7 illustrates the implemented user interface with screenshots and brief explanations.

2 ER Diagram

The complete ER diagram may be seen in Figure 1. Also, the actual PNG version of the diagram could be investigated further in the submission package with the naming of "`ER_Diagram.png`". Similarly, the diagram may also be reached through our publicly available GitHub repository (for more details, refer to Section 7).

A summary of the modelling choices is given below. The diagram follows Chen-style notation as taught in our lectures, using the following symbols:

- **Rectangles** represent entities; the double-bordered rectangle marks the weak entity, `Order_Status_History`.
- **Diamonds** represent relationships; the double-bordered diamond `Has_Status` marks the identifying relationship for the weak entity.

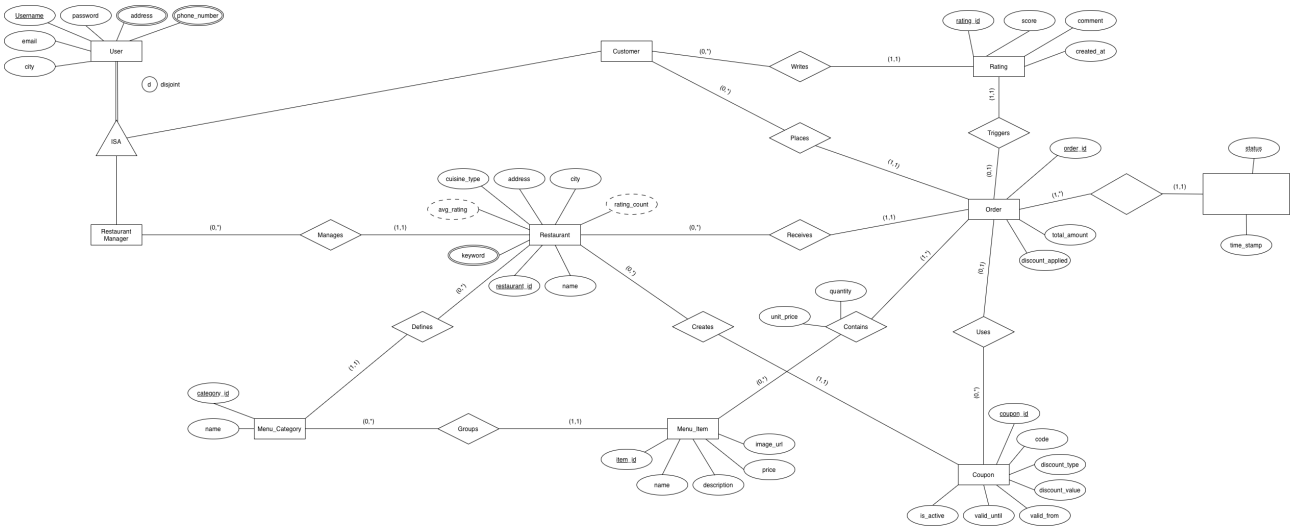

Figure 1: Chen-style entity-relationship (ER) diagram of the Online Food Ordering System, illustrating entities, relationships, specialization hierarchy, cardinalities, weak entities, and derived/multi-valued attributes.

- **Ellipses** represent attributes. **Double-bordered ellipses** represent multi-valued attributes (such as, `address` and `phone_number` on User; `keyword` on Restaurant). **Dashed ellipses** represent derived attributes (such as, `avg_rating` and `rating_count` on Restaurant).
- **Solid underlines** mark primary keys. The **dashed underline** under status on `Order_Status_History` marks the partial key (discriminator).
- The **ISA triangle** below User represents specialization into the subtypes Customer and Restaurant Manager. The letter **d** indicates that the subtypes are disjoint; the **double line** from User to the triangle indicates total participation (every user must be one of the two subtypes).
- **Cardinalities** are given using (min, max) notation, with the label placed on the same side as the entity it describes.

The diagram includes nine regular entities (User, Customer, Restaurant Manager, Restaurant, Menu_Category, Menu_Item, Order, Coupon, Rating), one weak entity (Order_Status_History), and eleven relationships (Manages, Places, Writes, Receives, Defines, Creates, Groups, Contains, Uses, Triggers, Has_Status). The Contains relationship between Order and Menu_Item is many-to-many and carries two relationship attributes: `quantity` and `unit_price`. Storing `unit_price` on the relationship rather than reading it from "Menu_Item.price" at query time records the price at the moment the order was placed, which preserves historical accuracy if the restaurant later updates its menu.

3 List of Functional Dependencies

Functional dependencies are listed by table. Each entry has the form $X \to Y$, meaning the values on the left side uniquely determine the values on the right.

User

- `username` $\to$ `password`, `email`, `city`

User_Address

- 1NF decomposition of the multi-valued `address` attribute
- Primary key (`username`, `address`); no non-trivial functional dependency beyond the key.

User_Phone

- (1NF decomposition of the multi-valued `phone_number` attribute)
- Primary key (`username`, `phone_number`); no non-trivial functional dependency beyond the key.

Restaurant

- `restaurant_id` → `name`, `cuisine_type`, `address`, `city`, `manager_id`

Restaurant_Keyword

- 1NF decomposition of the multi-valued `keyword` attribute
- Primary key (`restaurant_id`, `keyword`); no non-trivial functional dependency beyond the key.

Menu_Category

- `category_id` → `name`, `restaurant_id`

Menu_Item

- `item_id` → `name`, `description`, `price`, `image_url`, `category_id`

Order

- `order_id` → `customer_id`, `restaurant_id`, `coupon_id`, `status`, `created_at`, `total_amount`, `discount_applied`

Order_Item

- translation of the M:N `Contains` relationship
- `(order_id, item_id)` → `quantity`, `unit_price`

Order_Status_History

- weak entity
- `(order_id, status)` → `time_stamp`

Coupon

- `coupon_id` → `code`, `discount_type`, `discount_value`, `valid_from`, `valid_until`, `is_active`, `restaurant_id`

Rating

- `rating_id` → `score`, `comment`, `created_at`, `customer_id`, `order_id`

Each table's primary key functionally determines every non-key attribute in that table. No additional non-trivial dependencies exist among the non-key attributes of any table.

4 Explanation of Normalization (up to 3NF)

All tables in the resulting relational schema are in third normal form. The reasoning is presented below by normal form.

4.1 First Normal Form

1NF requires that every attribute be atomic — no cell may contain a list, set, or other composite value. Three attributes in the conceptual schema are multi-valued: `address` and `phone_number` on User, and `keyword` on Restaurant. In the relational schema these are not stored as comma-separated strings or arrays; instead they are decomposed into three separate tables — User_Address`(username, address)`, User_Phone`(username, phone_number)`, and Restaurant_Keyword`(restaurant_id, keyword)` — each with a composite primary key and no other columns. Every remaining attribute in every other table is a scalar value of a basic SQL type. All tables are therefore in 1NF.

4.2 Second Normal Form

2NF requires that every non-key attribute be fully functionally dependent on the entire primary key. The condition is only meaningful for tables with composite keys; in this schema those are `Order_Item`, `Order_Status_History`, and the three multi-valued decomposition tables.

In Order_Item`(order_id, item_id, quantity, unit_price)`, both non-key attributes depend on the full composite key. The choice to store `unit_price` on the line item rather than read it from Menu_Item.`price` warrants comment. If `unit_price` represented the current menu price, it would depend only on `item_id`, which would be a partial dependency and a 2NF violation. By design, however, `unit_price` represents the price of this item, in this specific order, captured at the moment the order was placed. The attribute therefore depends genuinely on the full key `(order_id, item_id)` and the table is in 2NF. This design also preserves the integrity of historical order totals when menu prices later change.

In Order_Status_History`(order_id, status, time_stamp)`, the only non-key attribute, `time_stamp`, records the moment at which the order reached the given status. Since the same order may reach multiple statuses at different times and the same status may occur in many different orders, `time_stamp` depends genuinely on both elements of the key.

The three multi-valued decomposition tables have no non-key attributes, so the 2NF condition is satisfied trivially. All composite-key tables are therefore in 2NF.

4.3 Third Normal Form

3NF requires that no non-key attribute depend transitively on the primary key through another non-key attribute. Each table was checked against this condition.

The two cases where a transitive dependency could plausibly have been introduced were deliberately avoided:

- Menu_Item does not contain a `restaurant_id` column. Every menu item belongs to a restaurant, but the relationship is mediated by `category_id`, which itself determines `restaurant_id`. Storing `restaurant_id` directly on Menu_Item would create the transitive chain `item_id` → `category_id` → `restaurant_id`. The restaurant of a menu item is instead reached through the join Menu_Item → Menu_Category → Restaurant.
- Rating does not contain a `restaurant_id` column. The restaurant being rated is determined by the order that triggered the rating: `rating_id` → `order_id` → `restaurant_id`. Storing `restaurant_id` on Rating would introduce the same kind of transitive dependency. Queries that need to associate ratings with restaurants join Rating to Order to Restaurant.

All other non-key attributes in every table depend directly on their respective primary keys. The schema is therefore in 3NF.

5 Key Design Decisions and Constraint Explanations

This section records the principal design choices made during conceptual and logical schema design.

5.1 Specialization of User into Customer and Restaurant Manager

User is specialized through an ISA hierarchy into two disjoint subtypes, Customer and Restaurant Manager. The specialization is total (every user must be one of the two subtypes — no third role exists in the system) and disjoint (no user can simultaneously be both). The constraint is enforced at registration time: the user selects exactly one role, and the corresponding subtype record is created together with the User record in a single transaction.

5.2 Weak Entity Order_Status_History

The three order stages defined in the specification (Preparing → Sent → Accepted) could have been represented as three "nullable" `timestamp` columns on the Order table. We chose instead to model status transitions as a weak entity, Order_Status_History. Each status change produces a row with the new status and the moment it occurred. The composite primary key `(order_id, status)` reflects the existence dependence of a status entry on its owning order, and the foreign key `order_id` uses ON DELETE CASCADE, so a status history row cannot survive its order. This design gives a complete audit trail of the order's life cycle and accommodates the future addition of new status types without schema changes.

5.3 Snapshot of unit_price on Order_Item

The attribute `unit_price` is stored on Order_Item rather than read from Menu_Item.`price` at query time. This decision serves two distinct purposes. First, it satisfies 2NF, as discussed in Section 4.2: `unit_price` depends on the full composite key `(order_id, item_id)` rather than on `item_id` alone.

Second, it preserves historical accuracy: when a restaurant later updates a menu item's price, past orders continue to show the price the customer actually paid.

5.4 Derived attributes avg_rating and rating_count

Restaurant carries two derived attributes shown in the ER diagram as dashed ellipses: `avg_rating` (the mean of ratings for orders received by the restaurant) and `rating_count` (the number of such ratings). Neither is stored as a column. Both are computed at query time by joining Rating through Order

to Restaurant. This avoids the risk of denormalized values becoming stale and keeps the Restaurant table in 3NF.

The "≤10 ratings to display the average" rule in the specification file of our project is implemented inside the search query through a CASE expression on `rating_count`; restaurants below the threshold are labelled "New" and reported with an average of zero.

5.5 Storage of Order.total_amount as deliberate denormalization

The attribute `total_amount` on Order is functionally derivable: it equals SUM(`quantity` × `unit_price`) over the order's line items, minus the applied coupon discount. It is nevertheless stored as a column. Two reasons motivate this choice.

First, the total is read in nearly every customer- and manager-facing query (order history, current order summary, monthly statistics) and recomputing it through aggregation on every read would noticeably degrade response time.

Second, storing the total snapshots the amount the customer was actually charged, decoupling it from any later modification of historical line items. This is a controlled denormalization, not a 3NF violation: `total_amount` does not depend on any non-key attribute of Order, and the system maintains the invariant Order.`total_amount` = SUM(Order_Item) - Order.`discount_applied` at the moment of insertion through transactional application logic.

5.6 Coupon and Restaurant matching

A coupon is created by exactly one restaurant through the Creates relationship and may be applied only to orders from that same restaurant. The constraint is not directly expressible in the ER diagram or in a CHECK constraint (MySQL does not allow subqueries in CHECK method). It is enforced at two levels: the Spring application validates the coupon's restaurant against the order's restaurant before allowing the coupon to be applied, and a BEFORE INSERT trigger on Order rejects any insert whose `coupon_id` references a coupon from a different restaurant. The two layers form a defense in depth — the application provides a clean user-facing error message, the trigger guarantees correctness even in the face of a programming error or direct database access.

5.7 Single-restaurant constraint on order items

The specification requires that all menu items in a single order come from the same restaurant. This is also not directly expressible in the conceptual schema. It is enforced in the application by the order-building logic, which only allows items from the same restaurant as items already in the cart, and at the database level by a trigger on Order_Item that rejects an insert if the item's restaurant (reached via `category_id` → `restaurant_id`) differs from the parent order's restaurant.

5.8 24-hour rating window

The specification allows customers to rate a restaurant only within 24 hours of the order being accepted. The constraint is enforced in application code: the rating submission endpoint queries Order_Status_History for the row where status = 'Accepted' and rejects the rating if more than 24 hours have elapsed since that timestamp. The constraint cannot be enforced cleanly through DDL because MySQL does not support subqueries in CHECK function.

5.9 Same-city visibility for customers

The specification requires that a customer can only see restaurants in their own city. This is enforced in the search query by filtering on Restaurant.`city` = User.`city` for the logged-in customer. The city attribute is stored on User directly, even though users may register multiple addresses through the User_Address table. The reasoning is that the customer's "default city" for filtering purposes is

a single value chosen at registration and changed deliberately, rather than being inferred from any particular delivery address.

5.10 "New" label for restaurants with fewer than ten ratings

Restaurants with fewer than ten ratings are labelled "New" in search results, and their average rating is reported as zero rather than the actual computed average. This is, in particular, vital not to "bias" the ratings of such new restaurants, where the reviews have not been accumulated enough.

This rule is implemented in the search query using a CASE expression on `rating_count`. We did not introduce a stored `is_new` column on Restaurant because that would risk inconsistency with the live count of ratings; the count is the authoritative source.

6 Main SQL Queries

The application's database interactions are implemented as parameterized SQL statements executed through JDBC. A representative selection is presented below. Each query is followed by a brief description of when it runs and what it returns.

Note: The queries shown here are illustrative of the application's required functionality. The exact column names and table names follow the schema defined in `DDL.sql`. Refer to Section 7 for code availability.

6.1 Restaurant search with keyword and city filter

Runs whenever a customer issues a restaurant search. Filters by the customer's city, joins keyword matches, aggregates ratings, and applies the "New" threshold.

```
SELECT r.restaurant_id, r.name, r.cuisine_type,
       COUNT(DISTINCT rk.keyword) AS keyword_matches,
       CASE WHEN COUNT(rt.rating_id) >= 10
            THEN AVG(rt.score)
            ELSE 0 END AS avg_rating,
       COUNT(rt.rating_id) AS rating_count,
       CASE WHEN COUNT(rt.rating_id) < 10 THEN 'New' ELSE NULL END AS label
FROM Restaurant r
LEFT JOIN Restaurant_Keyword rk
       ON rk.restaurant_id = r.restaurant_id
       AND rk.keyword LIKE ?
LEFT JOIN `Order` o
       ON o.restaurant_id = r.restaurant_id
LEFT JOIN Rating rt
       ON rt.order_id = o.order_id
WHERE r.city = ?
GROUP BY r.restaurant_id, r.name, r.cuisine_type
ORDER BY keyword_matches DESC, avg_rating DESC;
```

Listing 1: Restaurant recommendation query

6.2 Browse menu by category

Runs when a customer opens a restaurant's menu page. Returns categories and items.

```
SELECT mc.category_id, mc.name AS category_name,
       mi.item_id, mi.name AS item_name,
       mi.description, mi.price, mi.image_url
FROM Menu_Category mc
```

```
LEFT JOIN Menu_Item mi
       ON mi.category_id = mc.category_id
WHERE mc.restaurant_id = ?
ORDER BY mc.category_id, mi.item_id;
```

Listing 2: Menu retrieval query for a restaurant

6.3 Place an order

Runs when the customer confirms the cart. Executed as a single transaction so that an order is never persisted without its line items and initial status row.

```
-- Step 1: validate coupon (if one is being applied)
SELECT discount_type, discount_value
FROM Coupon
WHERE coupon_id = ?
  AND restaurant_id = ?
  AND is_active = TRUE
  AND CURRENT_TIMESTAMP BETWEEN valid_from AND valid_until;

-- Step 2: insert the Order row
INSERT INTO `Order`
  (customer_id, restaurant_id, coupon_id, status,
   created_at, total_amount, discount_applied)
VALUES (?, ?, ?, 'Preparing',
        CURRENT_TIMESTAMP, ?, ?);

-- Step 3: insert each line item
INSERT INTO Order_Item
  (order_id, item_id, quantity, unit_price)
VALUES (?, ?, ?, ?);

-- Step 4: log the initial status transition
INSERT INTO Order_Status_History
  (order_id, status, time_stamp)
VALUES (?, 'Preparing', CURRENT_TIMESTAMP);
```

Listing 3: Order placement and coupon validation workflow

6.4 Track order status with full history

Runs when a customer or manager views the progress of an order.

```
SELECT o.order_id,
       o.status AS current_status,
       o.total_amount,
       osh.status AS history_status,
       osh.time_stamp
FROM `Order` o
JOIN Order_Status_History osh
       ON osh.order_id = o.order_id
WHERE o.order_id = ?
ORDER BY osh.time_stamp ASC;
```

Listing 4: Order tracking and status history query

6.5 Restaurant accepts an incoming order

Runs when the restaurant manager clicks "Accept" on a pending order.

```sql
-- Update current status on Order
UPDATE `Order`
SET status = 'Accepted'
WHERE order_id = ?;

-- Append a new history row
INSERT INTO Order_Status_History
  (order_id, status, time_stamp)
VALUES (?, 'Accepted', CURRENT_TIMESTAMP);
```

Listing 5: Order acceptance and status history update workflow

6.6 Submit a rating (after 24-hour window check in application)

Application code first verifies that the order has been accepted within the last 24 hours; then the rating is inserted.

```sql
-- Step 1: read the acceptance timestamp
SELECT time_stamp
FROM Order_Status_History
WHERE order_id = ?
  AND status = 'Accepted';

-- application checks:
-- (NOW() - time_stamp) <= 24 hours

-- Step 2: insert the rating
INSERT INTO Rating
  (score, comment, created_at,
   customer_id, order_id)
VALUES (?, ?, CURRENT_TIMESTAMP,
        ?, ?);
```

Listing 6: Rating eligibility check and submission workflow

6.7 Monthly statistics — total revenue and order count

Runs when the restaurant manager opens the statistics dashboard.

```sql
SELECT COUNT(*) AS total_orders,
       COALESCE(SUM(total_amount), 0)
           AS total_revenue
FROM `Order`
WHERE restaurant_id = ?
  AND created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  );
```

Listing 7: Monthly restaurant order and revenue analytics query

6.8 Monthly statistics — item-wise quantity sold and revenue

Runs alongside Section 6.7 on the manager's statistics dashboard. Breaks down sales of the past month by individual menu item, reporting how many units of each item were sold and how much revenue each item generated. Results are ordered by revenue so that the highest-earning items appear first.

```
SELECT mi.item_id,
       mi.name,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit_price)
           AS item_revenue
FROM Order_Item oi
JOIN `Order` o
       ON o.order_id = oi.order_id
JOIN Menu_Item mi
       ON mi.item_id = oi.item_id
WHERE o.restaurant_id = ?
  AND o.created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  )
GROUP BY mi.item_id, mi.name
ORDER BY item_revenue DESC;
```

Listing 8: Monthly menu item sales and revenue analytics query

6.9 Monthly statistics — top customer by order count and by order value

Two related queries on the manager's statistics dashboard. The first identifies the customer who placed the largest number of orders in the past month — useful for recognising the restaurant's most loyal regulars. The second identifies the single highest-value order of the past month, surfaced together with the customer who placed it, the total amount, and the timestamp.

```
-- Most orders placed in the last month
SELECT o.customer_id,
       COUNT(*) AS order_count
FROM `Order` o
WHERE o.restaurant_id = ?
  AND o.created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  )
GROUP BY o.customer_id
ORDER BY order_count DESC
LIMIT 1;

-- Highest single order value in the last month
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.created_at
FROM `Order` o
WHERE o.restaurant_id = ?
  AND o.created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  )
```

```sql
ORDER BY o.total_amount DESC
LIMIT 1;
```

Listing 9: Monthly customer and order performance analytics query

6.10 Monthly statistics — most-ordered item, top-revenue category, total coupon discount

Three further aggregations on the manager's statistics dashboard. The first finds the single menu item with the highest total units sold in the past month. The second finds the menu category that contributed the largest share of revenue, joining line items through Menu_Item to Menu_Category. The third sums the discount amounts applied through coupons, showing the manager the total cost of their promotional activity over the past month.

```sql
-- Most-ordered item (by units)
SELECT mi.item_id,
       mi.name,
       SUM(oi.quantity) AS units_sold
FROM Order_Item oi
JOIN `Order` o
       ON o.order_id = oi.order_id
JOIN Menu_Item mi
       ON mi.item_id = oi.item_id
WHERE o.restaurant_id = ?
  AND o.created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  )
GROUP BY mi.item_id, mi.name
ORDER BY units_sold DESC
LIMIT 1;

-- Category with highest revenue
SELECT mc.category_id,
       mc.name,
       SUM(oi.quantity * oi.unit_price)
           AS category_revenue
FROM Order_Item oi
JOIN Menu_Item mi
       ON mi.item_id = oi.item_id
JOIN Menu_Category mc
       ON mc.category_id = mi.category_id
JOIN `Order` o
       ON o.order_id = oi.order_id
WHERE o.restaurant_id = ?
  AND o.created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  )
GROUP BY mc.category_id, mc.name
ORDER BY category_revenue DESC
LIMIT 1;

-- Total discount given through coupons
-- in the last month
SELECT COALESCE(
           SUM(discount_applied), 0
       ) AS total_coupon_discount
```

```
FROM `Order`
WHERE restaurant_id = ?
  AND coupon_id IS NOT NULL
  AND created_at >= DATE_SUB(
      CURRENT_DATE,
      INTERVAL 1 MONTH
  );
```

Listing 10: Monthly restaurant performance analytics query

7 Code availability

The complete source code for this project, together with the database scripts and supporting documentation, is available in a public GitHub repository:

Repository: https://github.com/USERNAME/REPOSITORY

The repository contains the Spring Boot backend, the Java UI source, the `DDL.sql` schema definition, the `DML.sql` sample-data script, the final ER diagram, and a `README.md` with setup and run instructions. The system can be reproduced locally by cloning the repository and following the steps in the `README`; default MySQL connection settings are configurable in `application.properties`.

8 Conclusion

The Online Food Ordering System was designed top-down, beginning with a conceptual entity-relationship model and translating it systematically into a normalized relational schema. The principal design choices — modelling order status transitions as a weak entity, snapshotting unit prices on line items, computing rating aggregates at query time rather than storing them, and enforcing business rules through a combination of database triggers and Spring application logic — reflect a deliberate balance between schema cleanliness and practical query performance.

The resulting database design satisfies third normal form, and all SQL was written manually rather than generated by an ORM, as strictly required by the project specification file provided. The application logic in the Spring backend communicates with the database through JDBC and exposes a Java UI that supports the full customer and restaurant manager workflows described in the specification.